

Wham — .NET Backend Construction Specification

ASP.NET Core 9 · PostgreSQL · Nethereum · Arbitrum

- Wham — .NET Backend Construction Specification
 - 0. Addendum — v1.1 integration corrections
 - 1. How to use this document
 - 2. Architecture
 - 3. Cross-cutting API conventions
 - 4. Domain model and persistence
 - 5. Authentication — SIWE
 - 6. Endpoint implementations
 - 7. The underwriting agent
 - 8. Chain integration
 - 9. Reputation engine
 - 10. Notifications
 - 11. Background services
 - 12. Testing
 - 13. Deployment
 - 14. Build order

Wham — .NET Backend Construction Specification

Version: 1.1 — includes the §0 integration addendum aligning this document with the smart contract spec **Companion to:** *Wham — Frontend Construction Specification v1.0* **Target:** Arbitrum Open House Singapore Online Buildathon (Sep 13 - Oct 4, 2026) **Scope:** Every service, entity, endpoint, background job, chain interaction, and scoring rule the backend needs — with the wire contracts pinned to the frontend spec so the two can be built in parallel.

0. Addendum — v1.1 integration corrections

The smart contract specification (v1.0) was finalized after this document and made several decisions that supersede sections below. **Where this addendum conflicts with the body, the addendum wins.** Everything not mentioned here stands.

0.1 Reputation is computed on-chain — §9 becomes projection and reconciliation

`WhamReputation` computes the score itself, from constants published in the contract (identical to the §9.1 table), fed by circles calling `recordContribution` / `recordMiss` / `recordDefault` / `recordCompletion` during settlement. Consequences:

- The indexer consumes `ScoreUpdated` and `AttestationMinted` events into `reputation_profiles` and `reputation_events` — the handler map in §8.7 already covers this. **Delete the recompute-from-events scoring logic in §9.3;** replace it with a reconciliation job that recomputes what the score *should* be and alerts on divergence from the chain. Divergence means an indexer bug, and trusting the chain while alerting is the correct response.
- The scoring engine in §7 (the **agent's** collateral scoring) is unaffected — that was never the reputation score.
- Tier computation (§9.2) can stay backend-side for query speed but must match `tierOf()` on-chain; add a test asserting the two agree across the threshold boundaries.

0.2 `join()` transfers collateral only

§6.5's `requirements` keeps its shape, but `totalDueNow` is the **approval total** (collateral + first contribution), not the on-chain transfer at join, which is collateral alone. The first contribution arrives via `contribute(0)` after `CircleStarted` — which means round 0 generates a normal contribution obligation and reminder like every other round. No special-casing.

`confirm-join` (§6.6) is unchanged: it watches for `MemberJoined` only.

0.3 `DeployParamsDto` gains two fields

Exact contract struct order (send it in this order; the frontend passes it through verbatim):

```

public sealed record DeployParamsDto(
    string Token, string ContributionPerRound, byte MemberTarget,
    int RoundDuration,
    int BiddingDuration, // NEW — roundDuration / 4; validate < RoundDuration
    ushort BaseCollateralBps, ushort InsuranceSkimBps, ushort MaxDiscountBps,
    ushort PayoutCollateralBps, // NEW — fixed at 5000 server-side, never client-supplied
    bool BiddingEnabled, long StartsAt, string Agent, string Salt);

```

link-deployment’s config verification (§6.4) must assert both new fields.

0.4 Enum mapping — do not cast

§4.3’s note that “the indexer casts directly” is retracted. The contract’s `MemberState` includes `None = 0`, shifting every subsequent ordinal by one relative to the backend enum. Maintain explicit mapping tables for all three shared enums with a test that asserts every contract ordinal maps, and fails on any unmapped value:

```

public static MemberStatus Map(byte onchain) => onchain switch
{
    1 => MemberStatus.PendingCollateral, 2 => MemberStatus.Active,
    3 => MemberStatus.PaidOut,           4 => MemberStatus.Delinquent,
    5 => MemberStatus.Defaulted,         6 => MemberStatus.Completed,
    7 => MemberStatus.Exited,
    _ => throw new UnmappedChainValueException("MemberState", onchain)
};

```

0.5 New and changed events to index

Additions to the §8.7 handler map:

Event	Projection effect
CircleStarted	Circle → Active, initialize round 0, schedule round-0 reminders
CircleCancelled	Circle → Cancelled, cancel reminders, notify members of refund availability
CollateralToppedUp	collateral_posted += amount on the winner; store on the round for the UI’s top-up row
RewardsWithdrawn	Decrement the member’s projected rewards balance
ShortfallUncovered	Record on the round; alert operations
CircleUnwinding	Circle → Unwinding, cancel all future reminders, notify all members
MemberDefaulted	Member → Defaulted (in addition to the CollateralDrawn path)

Changed: `ContributionPaid` now carries `bool onTime` — persist it (the projection no longer infers timeliness from timestamps), and `RoundSettled` carries `grossPot`. `MemberJoined` carries `joinIndex`; store it.

New projected field: `circle_members.pending_rewards (numeric(78,0))`, credited by `RoundSettled` redistribution, debited by `RewardsWithdrawn` and by the rewards-netting portion of contributions. Surface it in `/v1/me/summary` as `rewardsAvailable` — the frontend shows it on the dashboard.

0.6 The keeper also starts and cancels circles

`CircleStarter` (§11.2) does not flip a status flag — it sends transactions. When a `Forming` circle has `memberCount == memberTarget` and `startsAt` has passed and no `CircleStarted` event has arrived (the last joiner’s transaction usually auto-starts it, so this is a fallback), the keeper calls `start()`. When `startsAt + 7 days` passes without quorum, it calls `cancel()`. Same key and gas-price guard as `settleRound`.

0.7 Dropped logic

- **Bid tie handling** (§14.2 #3): ties are impossible on-chain — bids must improve by ≥ 25 bps. Delete the tiebreak code path.
- **§14.2 #1 (collateral top-up endpoint)**: not in contract v1; nothing to build.
- **§14.2 #4**: resolved by the payout-contingent top-up; the settlement projection must store the top-up amount per round

(from `CollateralToppedUp`) so `netPayout` and amount-received reconcile.

0.8 One addition to the EIP-712 test

The contract exposes `hashQuote(AgentQuote) → bytes32` as a public view. The backend integration test must assert its own computed digest equals the contract's for a fuzzed set of quotes — this is cheaper and stricter than the sign-and-join round trip alone, and it isolates encoding bugs from signature bugs.

1. How to use this document

The frontend spec defines **what the API returns**. This document defines **how it gets built**. Where the two disagree, the frontend spec's payload shapes win — they are the contract, and the frontend is being built against mocks derived from them.

Section mapping between the two documents:

This doc	Frontend spec
\$6 Endpoint implementations	\$7 Backend API reference
\$7 Underwriting agent	\$7.4 Agent + \$9.5 Agent hub
\$8 Chain integration	\$8 Smart contract interface
\$9 Reputation engine	\$6.8 + \$9.16
\$10 Notifications	\$6.9 + \$9.18

The single hardest thing in this backend is that it is not the source of truth. The contracts are. Every piece of circle state in PostgreSQL is a *projection* of on-chain events, kept for query speed, filtering, and notification scheduling. Build accordingly: the database can always be rebuilt by replaying the chain from the factory deployment block, and there should be a command that does exactly that.

2. Architecture

2.1 Responsibilities

The backend does five things the chain and the frontend cannot:

1. **Underwriting.** Compute a per-member collateral requirement, sign it EIP-712 so the contract will accept it, and attest the decision under the agent's ERC-8004 identity.
2. **Indexing.** Turn contract events into queryable state so `/circles?minContribution=...&sort=filling_fast` is one SQL query instead of forty RPC calls.
3. **Scheduling.** Know that a payment is due in 72 hours and tell someone. The chain cannot push.
4. **Discovery.** Circle matching, invite codes, search, recommendations.
5. **Keeping.** Call `settleRound` on time so a circle doesn't stall because nobody clicked a button.

Everything else — custody, ordering, slashing, payouts — belongs to the contracts and must never be duplicated here.

2.2 Stack

Concern	Choice	Notes
Runtime	.NET 9, ASP.NET Core	
API style	Minimal APIs with route groups + endpoint filters	Faster to write than controllers, and the endpoint surface is small
Persistence	PostgreSQL 16 + EF Core 9	<code>numeric(78,0)</code> for token amounts
Chain access	Nethereum 4.x	<code>Nethereum.Web3</code> , <code>Nethereum.Siwe</code> , <code>Nethereum.Signer.EIP712</code>
Background work	<code>IHostedService</code> + a small job table	Hangfire is optional; for 3 weeks the hosted services are less setup
Validation	FluentValidation	

Mapping	Hand-written mappers in <code>Api/Mapping</code>	No AutoMapper — the DTO shapes are pinned to the frontend spec and must be explicit
Logging	Serilog → console (JSON) + Seq in dev	
Tracing	OpenTelemetry, exported to console in dev	<code>traceId</code> goes in every error envelope
Email	Postmark or Resend	
Telegram	Bot API, long-poll in dev, webhook in prod	
Tests	xUnit + FluentAssertions + Testcontainers (Postgres) + Anvil for chain	
Secrets	<code>dotnet user-secrets</code> in dev, env vars in prod	

2.3 Solution layout

```

Wham.sln
├─ src/
│   ├─ Wham.Api/           # host: minimal API endpoints, DI wiring, middleware
│   │   ├─ Endpoints/      # one file per resource group
│   │   ├─ Contracts/     # request/response DTOs – mirrors frontend spec §6
│   │   ├─ Mapping/       # domain → DTO
│   │   ├─ Filters/       # auth, validation, error envelope
│   │   └─ Program.cs
│   ├─ Wham.Domain/       # entities, enums, domain services, no dependencies
│   │   ├─ Entities/
│   │   └─ Enums/
│   │       └─ ValueObjects/ # EvmAddress, TokenAmount, Bps
│   ├─ Wham.Infrastructure/ # EF Core, repositories, external clients
│   │   ├─ Persistence/
│   │   │   └─ WhamDbContext.cs
│   │   └─ Configurations/
│   │       └─ Migrations/
│   │   └─ Chain/         # Nethereum clients, event DTOs, ABIs
│   │   └─ Notifications/ # email + telegram senders
│   │       └─ Auth/      # SIWE, JWT
│   └─ Wham.Underwriting/ # the agent: rules, scoring, EIP-712 signing, attestation
│       ├─ Rules/
│       └─ ScoringEngine.cs
│           └─ QuoteSigner.cs
│               └─ AttestationPublisher.cs
│   └─ Wham.Indexer/      # hosted services: log polling, projection, reorg handling
│       └─ Wham.Workers/  # hosted services: reminders, keeper, lifecycle, outcomes
├─ tests/
│   ├─ Wham.Underwriting.Tests/ # table-driven, deterministic – the most valuable tests here
│   ├─ Wham.Api.Tests/         # WebApplicationFactory + Testcontainers
│   └─ Wham.Indexer.Tests/     # replay fixtures against a local Anvil chain
└─ deploy/
    ├─ docker-compose.yml
    └─ Dockerfile

```

Deployment shape: one container image, three roles selected by `WHAM_ROLE` (`api`, `indexer`, `workers`). In development run all three in-process (`WHAM_ROLE=all`) so `dotnet run` gives you a working system. In production split them so a slow indexer can't stall API requests, and so only the workers container holds the agent's signing key.

2.4 Configuration

```
// appsettings.json – non-secret defaults
{
  "Wham": {
    "ChainId": 421614,
    "RpcUrl": "https://sepolia-rollup.arbitrum.io/rpc",
    "RpcUrlFallback": "",
    "Contracts": {
      "CircleFactory": "0x1F4a2B9c3D8e5A7f0B1c2D3e4F5a6B7c8D9e0F1a",
      "Reputation": "0x2A5b3C0d4E9f6B8a1C2d3E4f5A6b7C8d9E0f1A2b",
      "InsurancePool": "0x3B6c4D1e5F0a7C9b2D3e4F5a6B7c8D9e0F1a2B3c",
      "AgentIdentity": "0x4C7d5E2f6A1b8D0c3E4f5A6b7C8d9E0f1A2b3C4d",
      "AgentReputation": "0x5D8e6F3a7B2c9E1d4F5a6B7c8D9e0F1a2B3c4D5e",
      "Usdc": "0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d"
    },
    "Indexer": {
      "StartBlock": 91450000,
      "PollIntervalSeconds": 4,
      "MaxBlockRange": 2000,
      "ConfirmationDepth": 5,
      "ReorgRewindBlocks": 30
    },
    "Agent": {
      "AgentId": "8004:421614:1042",
      "ModelVersion": "heuristic-v1.2.0",
      "ModelKind": "rules",
      "QuoteTtlMinutes": 15,
      "AttestationBatchSize": 20,
      "AttestationIntervalMinutes": 10
    },
    "Auth": {
      "Domain": "wham.finance",
      "NonceTtlMinutes": 10,
      "SessionTtlDays": 7,
      "Issuer": "https://api.wham.finance",
      "Audience": "wham-app"
    },
    "Keeper": {
      "Enabled": true,
      "SettleGraceMinutes": 5,
      "MaxGasPriceGwei": 2.0
    },
    "Reminders": {
      "DefaultLeadHours": [72, 24, 4],
      "DispatchIntervalMinutes": 5
    }
  }
}
```

Secrets — never in appsettings :

Key	Purpose
Wham:Agent:PrivateKey	Signs EIP-712 quotes and sends attestation transactions
Wham:Keeper:PrivateKey	Sends settleRound transactions. Separate key from the agent.
Wham:Auth:JwtSigningKey	32+ bytes
ConnectionStrings:Wham	
Wham:Email:ApiKey, Wham:Telegram:BotToken	

Key separation matters for the pitch. The agent key’s only on-chain power is signing collateral quotes and writing attestations. It cannot move funds, cannot settle rounds, cannot touch the insurance pool. Say this on the /agent page and

make it true in the deployment.

3. Cross-cutting API conventions

3.1 Error envelope

Every non-2xx response, without exception:

```
{
  "error": {
    "code": "QUOTE_EXPIRED",
    "message": "Your deposit quote expired. Get a fresh one and try again.",
    "details": { "decisionId": "d_5cle8a02", "expiredAt": "2026-09-27T19:00:00Z" },
    "traceId": "0af7651916cd43dd8448eb211c80319c"
  }
}
```

Implemented as an exception middleware over a `WhamException` hierarchy:

```
public abstract class WhamException(string code, string message, int status) : Exception(message)
{
    public string Code { get; } = code;
    public int StatusCode { get; } = status;
    public IDictionary<string, object?> Details { get; } = new Dictionary<string, object?>();
}

public sealed class CircleFullException(Guid circleId)
    : WhamException("CIRCLE_FULL", "This circle just filled up.", StatusCodes.Status409Conflict);

public sealed class QuoteExpiredException(string decisionId, DateTimeOffset expiredAt)
    : WhamException("QUOTE_EXPIRED",
        "Your deposit quote expired. Get a fresh one and try again.",
        StatusCodes.Status409Conflict);
```

Rule: `message` is user-facing copy and ships to production untouched. Write it as product copy, not as a developer log line. The frontend renders it verbatim (frontend spec §6.10). Never put an address, a stack trace, a SQL fragment, or a contract revert string in `message` — those go in `details`.

The full `ApiErrorCode` union is fixed by frontend spec §6.10. Adding a code is a contract change: update both documents.

3.2 Serialisation

```
options.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
options.DefaultIgnoreCondition = JsonIgnoreCondition.Never; // nulls are meaningful in this API
options.Converters.Add(new BigIntegerAsStringConverter());
options.Converters.Add(new JsonStringEnumConverter(JsonNamingPolicy.SnakeCaseLower));
options.Converters.Add(new UtcIso8601Converter()); // always "...Z", never offsets
```

Three non-negotiables:

1. **Token amounts serialise as decimal strings**, always. `BigInteger` → `"250000000"`. A `long` will hold USDC fine and then silently break the day someone uses an 18-decimal token.
2. **Nulls are emitted, not omitted**. The frontend's Zod schemas distinguish "absent" from "null" and `viewer.quotedCollateral: null` is load-bearing.
3. **Enums are snake_case strings** matching the frontend unions exactly: `pending_collateral`, not `PendingCollateral`.

3.3 Address handling

```
public readonly record struct EvmAddress
{
    private readonly string _lower; // storage + comparison form
    public string Value => _lower;
    public string Checksummed => new AddressUtil().ConvertToChecksumAddress(_lower);
    public static EvmAddress Parse(string s); // throws ValidationException on bad input
}
```

- **Store lowercase.** Index lowercase. Compare lowercase.
- **Return checksummed** in every DTO.
- EF value converter handles the round-trip; never hand-lowercase at a call site.

This one decision removes an entire class of “member not found” bugs that will otherwise eat an afternoon during demo prep.

3.4 Token amounts

```
public static readonly ValueConverter<BigInteger, decimal> BigIntConverter = new(
    v => (decimal)v,
    v => new BigInteger(v));
```

Map to `numeric(78,0)`. Never `bigint`, never `decimal` in the domain model, never a floating-point type anywhere in the process. All arithmetic on amounts uses `BigInteger`.

3.5 Pagination

```
public sealed record Paginated<T>(IEnumerable<T> Items, int Page, int PageSize, int Total, bool HasMore);
```

Default `pageSize` 12 for circles, 20 for activity and notifications. Cap at 100. Page numbers, not cursors — the datasets here are small and shareable URLs matter more than deep-scroll performance.

3.6 Rate limits

Endpoint group	Limit
POST /v1/auth/verify	10/min per IP
GET /v1/auth/nonce	20/min per IP
POST /v1/agent/quote	10/min per address, 60/hour
POST /v1/circles	5/hour per address
POST /v1/circles/{id}/join-intent	20/hour per address
Everything else	120/min per address or IP

Built on ASP.NET Core’s built-in `RateLimiter` with a fixed window. Exceeded → 429 with code `RATE_LIMITED` and a `Retry-After` header.

3.7 Caching

Endpoint	Cache
GET /v1/stats	Output cache 60s, shared
GET /v1/agent/identity	Output cache 60s, shared
GET /v1/agent/stats	Output cache 300s, shared
GET /v1/insurance/pool	Output cache 60s, shared
GET /v1/circles (unauthenticated)	Output cache 30s, varies by query string
GET /v1/circles (authenticated)	No cache — <code>viewer</code> context is per-address
Everything authenticated	<code>Cache-Control: no-store</code>

The authenticated/anonymous split on `/v1/circles` is a real trap: cache it naively and one user sees another user’s `viewer`

block. Vary the cache policy on the presence of the auth cookie, and write a test for it.

4. Domain model and persistence

Fourteen tables. Amounts are `numeric(78,0)`, addresses are `citext` or `varchar(42)` stored lowercase, timestamps are `timestampz` stored UTC.

4.1 Entity overview

Table	Source of truth	Purpose
users	Backend	Off-chain profile, preferences, contact channels
auth_nonces	Backend	SIWE nonce issue/consume
circles	Chain (after deploy)	Circle config + projected state
circle_members	Chain	Membership + collateral projection
rounds	Chain	Per-round state projection
contributions	Chain	Individual payments
bids	Chain	Bid history
agent_decisions	Backend	Underwriting decisions + signatures
seat_reservations	Backend	15-minute holds during the join flow
reputation_profiles	Derived	Materialised score per address
reputation_events	Derived	Score history
insurance_claims	Chain	Pool draws
notifications	Backend	In-app feed + delivery record
scheduled_reminders	Backend	Future sends
chain_events	Chain	Raw log ledger, idempotency guard
indexer_checkpoints	Backend	Per-stream block cursor

Anything marked **Chain** is rebuildable. Anything marked Backend is not — back it up.

4.2 Core entities

```
public sealed class User
{
    public string Address { get; set; } = default!; // PK, lowercase
    public string? DisplayName { get; set; }
    public string? Email { get; set; }
    public bool EmailVerified { get; set; }
    public string? EmailVerificationToken { get; set; }
    public long? TelegramChatId { get; set; }
    public string? TelegramHandle { get; set; }
    public string Timezone { get; set; } = "UTC"; // IANA
    public string Locale { get; set; } = "en"; // en | fa
    public bool FirstLogin { get; set; } = true;
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset LastSeenAt { get; set; }
    public NotificationPreferences Preferences { get; set; } = new(); // jsonb
}

public sealed class NotificationPreferences
{
    public int[] NotifyPaymentDueHours { get; set; } = [72, 24, 4];
    public bool NotifyBidWindow { get; set; } = true;
    public bool NotifyOutbid { get; set; } = true;
    public bool NotifyRoundSettled { get; set; } = true;
    public bool NotifyPayoutReady { get; set; } = true;
    public bool NotifyCollateralDrawn { get; set; } = true;
    public bool ChannelEmail { get; set; }
    public bool ChannelTelegram { get; set; }
    public bool ChannelInApp { get; set; } = true;
}
```

```

public sealed class Circle
{
    public Guid Id { get; set; } // PK; public id is "c_" + first 8 hex
    public string PublicId { get; set; } = default!; // "c_2e81aa93", unique, indexed
    public string? OnchainAddress { get; set; } // null until deploy is linked
    public int ChainId { get; set; }
    public string Name { get; set; } = default!;
    public string Description { get; set; } = "";
    public CircleStatus Status { get; set; }
    public CircleVisibility Visibility { get; set; }
    public string? InviteCode { get; set; } // "WHAM-4KQ2-8XLP", unique

    public string TokenAddress { get; set; } = default!;
    public BigInteger ContributionPerRound { get; set; }
    public byte MemberTarget { get; set; }
    public int RoundDurationSeconds { get; set; }
    public ushort BaseCollateralBps { get; set; }
    public ushort InsuranceSkimBps { get; set; }
    public ushort MaxDiscountBps { get; set; }
    public bool BiddingEnabled { get; set; }
    public CircleTier Tier { get; set; }
    public int MinReputationScore { get; set; }

    public byte? CurrentRoundIndex { get; set; }
    public DateTimeOffset? StartsAt { get; set; }
    public DateTimeOffset? EndsAt { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public string CreatedBy { get; set; } = default!;
    public string? DeployTxHash { get; set; }
    public string? Salt { get; set; }

    // Projected aggregates, recomputed by the indexer – never trusted as authoritative
    public byte MemberCount { get; set; }
    public byte RoundsSettled { get; set; }
    public BigInteger TotalDistributed { get; set; }
    public BigInteger InsuranceDrawn { get; set; }

    public List<CircleMember> Members { get; set; } = [];
    public List<Round> Rounds { get; set; } = [];
}

```

```

public sealed class CircleMember
{
    public Guid Id { get; set; }
    public Guid CircleId { get; set; }
    public string Address { get; set; } = default!;
    public MemberStatus Status { get; set; }
    public DateTimeOffset JoinedAt { get; set; }

    public BigInteger CollateralRequired { get; set; }
    public BigInteger CollateralPosted { get; set; }
    public BigInteger CollateralSlashed { get; set; }
    public DateTimeOffset? CollateralReleasedAt { get; set; }

    public byte RoundsPaid { get; set; }
    public byte RoundsMissed { get; set; }
    public sbyte PayoutRoundIndex { get; set; } = -1;
    public BigInteger? PayoutAmountNet { get; set; }
    public ushort? WinningDiscountBps { get; set; }
    public string? AgentDecisionId { get; set; }
    public byte RiskBandAtJoin { get; set; }
}

```

```

public sealed class Round
{
    public Guid Id { get; set; }
    public Guid CircleId { get; set; }
    public byte Index { get; set; }
    public RoundStatus Status { get; set; }

    public DateTimeOffset BiddingOpensAt { get; set; }
    public DateTimeOffset BiddingClosesAt { get; set; }
    public DateTimeOffset ContributionDueAt { get; set; }
    public DateTimeOffset? SettledAt { get; set; }

    public BigInteger GrossPot { get; set; }
    public BigInteger InsuranceSkim { get; set; }
    public string? WinnerAddress { get; set; }
    public ushort? WinningDiscountBps { get; set; }
    public BigInteger? DiscountAmount { get; set; }
    public BigInteger? NetPayout { get; set; }
    public BigInteger? RedistributionPerMember { get; set; }

    public byte ContributionsReceived { get; set; }
    public byte ContributionsExpected { get; set; }
    public BigInteger Shortfall { get; set; }
    public BigInteger CollateralDrawn { get; set; }
    public BigInteger InsuranceDrawnAmount { get; set; }
    public string? SettlementTxHash { get; set; }

    // Keeper bookkeeping – backend-owned, not projected
    public DateTimeOffset? SettleAttemptedAt { get; set; }
    public int SettleAttempts { get; set; }
    public string? SettleFailureReason { get; set; }
}

```

```

public sealed class AgentDecision
{
    public string Id { get; set; } = default!;           // "d_5c1e8a02"
    public string Subject { get; set; } = default!;
    public Guid CircleId { get; set; }
    public string? CircleAddress { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }

    public byte RiskBand { get; set; }                 // 0=A ... 3=D
    public ushort BaselineMultiplierBps { get; set; }
    public ushort FinalMultiplierBps { get; set; }
    public BigInteger CollateralRequired { get; set; }
    public List<AgentFactor> Factors { get; set; } = []; // jsonb

    public string AgentId { get; set; } = default!;
    public string ModelVersion { get; set; } = default!;
    public BigInteger Nonce { get; set; }
    public string Signature { get; set; } = default!;   // EIP-712, 0x-prefixed
    public string? AttestationTxHash { get; set; }
    public DateTimeOffset? AttestedAt { get; set; }

    public DecisionOutcome Outcome { get; set; } = DecisionOutcome.Pending;
    public DateTimeOffset? OutcomeResolvedAt { get; set; }

    /// Snapshot of the inputs at decision time – required for auditability.
    /// Without this you cannot explain a decision after the member's history moves on.
    public ScoringSnapshot InputSnapshot { get; set; } = new(); // jsonb
}

public sealed class ScoringSnapshot
{
    public int ReputationScore { get; set; }
    public int CirclesCompleted { get; set; }
    public int RoundsOnTime { get; set; }
    public int RoundsLate { get; set; }
    public int RoundsMissed { get; set; }
    public int Defaults { get; set; }
    public int ActiveCircles { get; set; }
    public int WalletAgeDays { get; set; }
    public BigInteger LargestPotCompleted { get; set; }
    public BigInteger CirclePotPerRound { get; set; }
    public DateTimeOffset? LastLatePaymentAt { get; set; }
}

```

The `InputSnapshot` field is the difference between an auditable agent and a black box. A decision made in September must still be explainable in December after the member has completed three more circles. Persist the inputs, not just the outputs.

```

public sealed class SeatReservation
{
    public Guid Id { get; set; }
    public Guid CircleId { get; set; }
    public string Address { get; set; } = default!;
    public string DecisionId { get; set; } = default!;
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }
    public DateTimeOffset? ReleasedAt { get; set; }
    public DateTimeOffset? ConsumedAt { get; set; }
    public string? JoinTxHash { get; set; }
}

```

```

public sealed class ChainEvent
{
    public long Id { get; set; }
    public int ChainId { get; set; }
    public string TxHash { get; set; } = default!;
    public int LogIndex { get; set; }
    public long BlockNumber { get; set; }
    public string BlockHash { get; set; } = default!; // for reorg detection
    public string ContractAddress { get; set; } = default!;
    public string EventName { get; set; } = default!;
    public string PayloadJson { get; set; } = default!;
    public DateTimeOffset BlockTimestamp { get; set; }
    public DateTimeOffset ProcessedAt { get; set; }
}

```

Unique index on `(chain_id, tx_hash, log_index)`. This is the idempotency guard for the entire indexer — every projection write happens in the same transaction as the `chain_events` insert, so a replay is a no-op.

4.3 Enums

Persist as `int`. Serialise as `snake_case` strings matching the frontend unions exactly.

```

public enum CircleStatus { Forming = 0, Active = 1, Completed = 2, Unwinding = 3, Cancelled = 4 }
public enum MemberStatus { PendingCollateral = 0, Active = 1, PaidOut = 2, Delinquent = 3,
    Defaulted = 4, Completed = 5, Exited = 6 }
public enum RoundStatus { Upcoming = 0, Bidding = 1, Funding = 2, Settled = 3, Covered = 4 }
public enum CircleTier { Starter = 0, Standard = 1, Trusted = 2 }
public enum CircleVisibility { Public = 0, Invite = 1 }
public enum DecisionOutcome { Pending = 0, CompletedClean = 1, MissedPayments = 2,
    Defaulted = 3, Expired = 4 }
public enum ReputationEventKind { CircleCompleted, RoundPaidOnTime, RoundPaidLate,
    RoundMissed, Default, TierUnlocked, AttestationMinted }

```

Note the enum ordinals must match the Solidity enums for `CircleStatus`, `MemberStatus`, and `RoundStatus` (contract spec §8.2). The indexer casts directly. If the contract adds a state, both sides change together — write a test that asserts the mapping table rather than relying on ordinals lining up by luck.

4.4 Indexes

```

CREATE UNIQUE INDEX ix_circles_public_id      ON circles (public_id);
CREATE UNIQUE INDEX ix_circles_onchain        ON circles (chain_id, onchain_address)
WHERE onchain_address IS NOT NULL;
CREATE UNIQUE INDEX ix_circles_invite         ON circles (invite_code) WHERE invite_code IS NOT NULL;
CREATE INDEX ix_circles_browse               ON circles (status, starts_at)
INCLUDE (contribution_per_round, member_target, tier);

CREATE UNIQUE INDEX ix_members_circle_addr    ON circle_members (circle_id, address);
CREATE INDEX ix_members_address              ON circle_members (address, status);

CREATE UNIQUE INDEX ix_rounds_circle_index    ON rounds (circle_id, index);
CREATE INDEX ix_rounds_due                   ON rounds (status, contribution_due_at);
CREATE INDEX ix_rounds_bidding_close         ON rounds (status, bidding_closes_at);

CREATE UNIQUE INDEX ix_contrib_unique         ON contributions (circle_id, round_index, address);
CREATE INDEX ix_bids_round                   ON bids (circle_id, round_index, discount_bps DESC);

CREATE INDEX ix_decisions_subject             ON agent_decisions (subject, created_at DESC);
CREATE INDEX ix_decisions_outcome            ON agent_decisions (outcome, risk_band);

CREATE INDEX ix_reservations_live            ON seat_reservations (circle_id)
WHERE released_at IS NULL AND consumed_at IS NULL;

CREATE UNIQUE INDEX ix_chain_events_unique    ON chain_events (chain_id, tx_hash, log_index);
CREATE INDEX ix_chain_events_block           ON chain_events (chain_id, block_number DESC);

CREATE INDEX ix_reminders_pending            ON scheduled_reminders (send_at)
WHERE sent_at IS NULL AND cancelled_at IS NULL;
CREATE INDEX ix_notifications_user           ON notifications (address, created_at DESC);

```

The partial index on `seat_reservations` is what makes the “is there a free seat?” check cheap, and that check runs on every join-intent under contention. Don’t skip it.

4.5 Capacity accounting — the one real concurrency problem

A circle with `memberTarget = 8` and 7 confirmed members has one seat. Two users hitting `join-intent` simultaneously must not both get a reservation, or one pays gas for a transaction that reverts with `CircleFull()`.

```

// Inside a serializable transaction
await using var tx = await db.Database.BeginTransactionAsync(IsolationLevel.Serializable, ct);

var circle = await db.Circles
    .FromSqlInterpolated($"SELECT * FROM circles WHERE id = {circleId} FOR UPDATE")
    .SingleAsync(ct);

var confirmed = await db.CircleMembers.CountAsync(m =>
    m.CircleId == circleId && m.Status != MemberStatus.Exited, ct);

var held = await db.SeatReservations.CountAsync(r =>
    r.CircleId == circleId && r.ReleasedAt == null && r.ConsumedAt == null
    && r.ExpiresAt > now, ct);

if (confirmed + held >= circle.MemberTarget)
    throw new CircleFullException(circleId);

```

`FOR UPDATE` on the circle row serialises contenders. Retry once on `40001` (serialisation failure) before surfacing the error — Npgsql surfaces these as `PostgresException` with `SqlState == "40001"`.

Reservations are advisory, not authoritative. The contract is the real gate: it will revert if the circle is genuinely full regardless of what this table says. The reservation exists to stop users wasting gas, not to enforce capacity.

5. Authentication — SIWE

5.1 Packages

```
Nethereum.Siwe  
Nethereum.Siwe.Core  
Microsoft.AspNetCore.Authentication.JwtBearer
```

5.2 Nonce issue and consume

```
app.MapGet("/v1/auth/nonce", async (WhamDbContext db, IOptions<WhamOptions> opts, CancellationToken ct) =>  
{  
    var nonce = RandomNumberGenerator.GetString(  
        "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz23456789", 20);  
  
    var now = DateTimeOffset.UtcNow;  
    db.AuthNonces.Add(new AuthNonce  
    {  
        Nonce = nonce,  
        IssuedAt = now,  
        ExpiresAt = now.AddMinutes(opts.Value.Auth.NonceTtlMinutes)  
    });  
    await db.SaveChangesAsync(ct);  
  
    return Results.Ok(new NonceResponse(nonce, now, now.AddMinutes(opts.Value.Auth.NonceTtlMinutes)));  
});
```

Nonces are single-use rows, not an in-memory cache. `InMemorySessionNonceStorage` from `Nethereum.Siwe` breaks the moment you run two API replicas, and you will run two.

5.3 Verify

```

app.MapPost("/v1/auth/verify", async (
    VerifyRequest req, WhamDbContext db, ISiweVerifier siwe,
    IJwtIssuer jwt, IOptions<WhamOptions> opts, CancellationToken ct) =>
{
    var message = SiweMessageParser.Parse(req.Message);

    // 1. Nonce: exists, unconsumed, unexpired – consume atomically
    var rows = await db.AuthNonces
        .Where(n => n.Nonce == message.Nonce && n.ConsumedAt == null && n.ExpiresAt > DateTimeOffset.UtcNow)
        .ExecuteUpdateAsync(s => s.SetProperty(n => n.ConsumedAt, DateTimeOffset.UtcNow), ct);
    if (rows == 0) throw new NonceInvalidException();

    // 2. Domain must match ours – prevents a signature farmed on another site being replayed here
    if (!string.Equals(message.Domain, opts.Value.Auth.Domain, StringComparison.OrdinalIgnoreCase))
        throw new SignatureInvalidException("This sign-in request came from an unexpected domain.");

    // 3. Chain must be one we serve
    if (message.ChainId != opts.Value.ChainId.ToString())
        throw new SignatureInvalidException("Switch to Arbitrum to sign in.");

    // 4. Expiry inside the message itself
    if (message.ExpirationTime is not null &&
        DateTimeOffset.Parse(message.ExpirationTime) < DateTimeOffset.UtcNow)
        throw new SignatureInvalidException("This sign-in request expired. Try again.");

    // 5. Signature recovery
    if (!await siwe.IsSignatureValidAsync(message, req.Signature))
        throw new SignatureInvalidException("We couldn't verify that signature.");

    var address = EvmAddress.Parse(message.Address);
    var user = await db.Users.FindAsync([address.Value], ct) ?? await CreateUserAsync(db, address, ct);
    user.LastSeenAt = DateTimeOffset.UtcNow;
    await db.SaveChangesAsync(ct);

    var (token, expiresAt) = jwt.Issue(address, opts.Value.Auth.SessionTtlDays);
    return Results.Ok(new VerifyResponse(token, expiresAt, user.ToProfileDto()));
});

```

The five checks are all load-bearing. Signature recovery alone proves someone signed *something*; it does not prove they signed **this** login, on **this** domain, for **this** chain, recently. Skipping checks 2–4 is the standard SIWE implementation bug.

Smart-contract wallets: `ISiweVerifier` should fall back to EIP-1271 (`isValidSignature(bytes32,bytes)` against the claimed address) when ECDSA recovery fails. Diaspora users on Safe or Argent will otherwise be locked out. Nethereum can call this directly; it's twenty lines and worth having.

5.4 JWT claims

```

new Claim(JwtRegisteredClaimNames.Sub, address.Value), // lowercase address
new Claim("addr", address.Checkedsummed),
new Claim("chain", chainId.ToString()),
new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())

```

HS256, 7-day expiry, issuer and audience validated. The Next.js BFF stores this in an `httpOnly` cookie (frontend spec §3.1) and it arrives as a bearer token via the same-origin rewrite.

5.5 The caller-identity filter

```

public static IResult RequireSelf(this ClaimsPrincipal user, string routeAddress)
    => string.Equals(user.FindFirstValue(JwtRegisteredClaimNames.Sub),
        routeAddress.ToLowerInvariant(), StringComparison.Ordinal)
        ? Results.Empty
        : throw new WhamForbiddenException("ADDRESS_MISMATCH", "That isn't your address.");

```

Every `/me/*` endpoint derives the address from the JWT and **never** from a route parameter or body field. Every circle-scoped mutation checks membership against the JWT address. This is the whole authorisation model; there are no roles.

6. Endpoint implementations

Payload shapes are fixed by frontend spec §7. This section covers the logic behind each.

6.1 GET `/v1/circles`

Filters: status, tier, contribution range, member target, round duration, `eligibleOnly`, `sort`, page.

```

var q = db.Circles.AsNoTracking().Where(c => c.Visibility == CircleVisibility.Public);

if (statuses.Length > 0) q = q.Where(c => statuses.Contains(c.Status));
if (minContribution is not null) q = q.Where(c => c.ContributionPerRound >= minContribution);
if (eligibleOnly && caller is not null)
    q = q.Where(c => c.MinReputationScore <= callerScore
        && c.MemberCount < c.MemberTarget
        && c.Status == CircleStatus.Forming);

q = sort switch
{
    "filling_fast" => q.OrderByDescending(c => (double)c.MemberCount / c.MemberTarget)
        .ThenBy(c => c.StartsAt),
    "pot_desc"    => q.OrderByDescending(c => c.ContributionPerRound * c.MemberTarget),
    "newest"      => q.OrderByDescending(c => c.CreatedAt),
    _             => q.OrderBy(c => c.StartsAt ?? DateTimeOffset.MaxValue)
};

```

Viewer context is computed in one batched pass after the page is materialised, never per-row:

```

public async Task<Dictionary<Guid, ViewerContextDto>> BuildAsync(
    IReadOnlyList<Circle> circles, string? caller, CancellationToken ct)
{
    if (caller is null) return [];

    var ids = circles.Select(c => c.Id).ToArray();
    var memberships = await db.CircleMembers
        .Where(m => ids.Contains(m.CircleId) && m.Address == caller)
        .ToDictionaryAsync(m => m.CircleId, ct);

    var profile = await reputation.GetProfileAsync(caller, ct);
    var balance = await chain.GetTokenBalanceAsync(caller, ct); // cached 30s per address
    var hasOpenDelinquency = await db.CircleMembers.AnyAsync(m =>
        m.Address == caller && m.Status == MemberStatus.Delinquent, ct);

    return circles.ToDictionary(c => c.Id, c => Compute(c, memberships.GetValueOrDefault(c.Id),
        profile, balance, hasOpenDelinquency));
}

```

`cannotJoinReason` is evaluated in a fixed precedence order, and the order is user-facing — the frontend renders one reason, so it must be the most useful one:

```
already_member → circle_started → circle_full → reputation_too_low
→ open_delinquency_elsewhere → insufficient_balance → null
```

`insufficient_balance` last, because it's the only one the user can fix in five minutes.

6.2 GET /v1/circles/{publicId}

Single query with `Include(c => c.Members).Include(c => c.Rounds)`, plus viewer context. `stats.onTimePaymentRate` is computed from `contributions` joined against `rounds`, cached on the circle row by the indexer rather than recomputed per request.

6.3 POST /v1/circles — create draft

1. Validate against the caller's tier caps:

Tier	Max members	Max contribution	Can create
Starter	8	100 USDC	Starter only
Standard	12	500 USDC	Starter, Standard
Trusted	20	unlimited	All

2. Generate `PublicId` (`"c_" + Guid.ToString("N")[..8]`), invite code if private, and a random 32-byte `salt`.
3. Derive `baseCollateralBps` from tier (starter 20000, standard 15000, trusted 12500) — the creator picks within a range, the tier sets the floor.
4. Persist with `Status = Forming`, `OnchainAddress = null`.
5. Return `deployParams` **exactly** as the contract's `CircleParams` struct field order (contract spec §8.1). The frontend passes this straight through to `createCircle`; a reordered field is a silent misconfiguration that only surfaces after deployment.

```
public sealed record DeployParamsDto(
    string Token, string ContributionPerRound, byte MemberTarget, int RoundDuration,
    ushort BaseCollateralBps, ushort InsuranceSkimBps, ushort MaxDiscountBps,
    bool BiddingEnabled, long StartsAt, string Agent, string Salt);
```

Note `agent` is included: the circle is deployed bound to the agent address that will sign its quotes. Read it from config, never accept it from the request.

6.4 POST /v1/circles/{id}/link-deployment

Body { `txHash`, `onchainAddress` }.

1. Fetch the receipt via Nethereum. Not found yet → return { `status: "pending"` } and let the frontend poll.
2. Decode the `CircleCreated` event from the receipt logs.
3. Assert `event.circle == onchainAddress`, `event.creator == caller`, `event.salt == circle.Salt`. Any mismatch → `VALIDATION_FAILED`. Do not trust the body.
4. Read `config()` from the deployed contract and assert it matches the stored draft field by field. **A user can deploy a circle with different parameters than they requested.** Catching that here is the difference between a trustworthy listing and a lie.
5. Set `OnchainAddress`, `DeployTxHash`, and register the address with the indexer's watch set.

6.5 POST /v1/circles/{id}/join-intent

The orchestration endpoint. Sequence:

1. Reject if: already a member, circle not Forming, reputation below minimum, open delinquency elsewhere
2. Serializable txn: check capacity (§4.5), insert `SeatReservation` (15 min TTL)
3. Call the scoring engine → `AgentDecision`
4. Sign the EIP-712 quote with the agent key
5. Persist the decision (including `InputSnapshot`) and link it to the reservation
6. Read the caller's on-chain balance and allowance
7. Return { `reservationId`, `expiresAt`, `decision`, `quote`, `requirements` }

If step 3 or 4 fails, release the reservation before throwing — otherwise a flaky agent leaks seats until the expiry sweeper

catches up.

`requirements` is computed, not stored:

```
var totalDueNow = decision.CollateralRequired + circle.ContributionPerRound;
new RequirementsDto(
    CollateralRequired: decision.CollateralRequired,
    FirstContribution: circle.ContributionPerRound,
    TotalDueNow:      totalDueNow,
    WalletBalance:    balance,
    SufficientBalance: balance >= totalDueNow,
    CurrentAllowance: allowance,
    ApprovalNeeded:   allowance >= totalDueNow ? BigInteger.Zero : totalDueNow);
```

6.6 POST /v1/circles/{id}/confirm-join

Body { `txHash`, `reservationId` }. Does **not** write membership — the indexer does, from the `MemberJoined` event. This endpoint only:

1. Records `JoinTxHash` on the reservation and marks it consumed
2. Polls the receipt up to 30s
3. Returns { `status`: "pending" | "confirmed" | "failed", `member` }

If the receipt shows a revert, decode the custom error and map it to user copy (contract spec §8.3), release the reservation, and return `failed` with that message.

Never write projected state from an HTTP handler. Every circle mutation reaches the database through exactly one path: the indexer. This rule is what makes the projection rebuildable, and breaking it once will produce a divergence you find during the demo.

6.7 GET /v1/me/summary

The dashboard payload. Assembled from parallel queries — it is the most-hit authenticated endpoint, so keep it to one round trip per source:

```
var (circles, obligations, profile, balance, unread) = await (
    db.CircleMembers.Where(m => m.Address == me && Active(m.Status)).Include(m => m.Circle).ToListAsync(ct),
    obligationService.GetUpcomingAsync(me, DateTimeOffset.UtcNow.AddDays(45), ct),
    reputationService.GetProfileAsync(me, ct),
    chain.GetTokenBalanceAsync(me, ct),
    db.Notifications.CountAsync(n => n.Address == me && n.ReadAt == null, ct)
).WhenAll();
```

`nextObligation` = the earliest unpaid obligation across all circles. `openBidWindows` = rounds in `Bidding` where the caller is a member and hasn't been paid out, with their bid and the leading bid.

6.8 Obligation generation

Obligations are **derived, never stored**. Computing them from rounds and contributions on each request keeps them correct after any chain event without a reconciliation job.

```

foreach (var membership in activeMemberships)
foreach (var round in membership.Circle.Rounds.Where(r => r.Status != RoundStatus.Settled))
{
    if (!HasContributed(membership.Address, round))
        yield return new Obligation(
            Type: ObligationType.Contribution,
            Amount: circle.ContributionPerRound,
            DueAt: round.ContributionDueAt,
            AutoDeductRisk: true,
            AutoDeductAmount: circle.ContributionPerRound,
            Status: Classify(round.ContributionDueAt, now));

    if (round.Status == RoundStatus.Bidding && membership.PayoutRoundIndex < 0)
        yield return new Obligation(Type: ObligationType.BidWindow, Amount: null,
            DueAt: round.BiddingClosesAt, AutoDeductRisk: false, ...);
}

if (membership.Status == MemberStatus.Completed && membership.CollateralReleasedAt is null)
    yield return new Obligation(Type: ObligationType.CollateralRelease,
        Amount: membership.CollateralPosted - membership.CollateralSlashed,
        DueAt: circle.EndsAt!.Value, AutoDeductRisk: false, ...);

```

Classify: > 72h → upcoming, ≤ 72h → due_soon, past due → overdue.

6.9 GET /v1/stats

Cached 60s, computed by a background job into a single-row table rather than aggregated per request.

```

MembersWhoLostFunds = await db.CircleMembers.CountAsync(m =>
    m.Status == MemberStatus.Completed
    && m.PayoutAmountNet != null
    && (m.CollateralPosted - m.CollateralSlashed + m.PayoutAmountNet)
    < ContributionsPaidBy(m), ct);

```

This number appears on the landing page as the headline trust claim (frontend spec §9.1). Compute it honestly, from actual member ledgers, and let it be non-zero if it ever is. A number you'd fudge is a number that isn't worth showing.

7. The underwriting agent

This is the differentiator. Everything else in the backend is competent plumbing; this section is what the submission is judged on. Build it so that a judge can read the rules, watch a decision get made, follow it on-chain, and check it against an outcome.

7.1 Design constraints

1. **Deterministic.** Same inputs, same version → same output, always. Non-determinism makes the on-chain record meaningless.
2. **Enumerable.** The rule set is data, not control flow, so `/v1/agent/identity` can publish it and the UI can render it as a table.
3. **Explainable per-factor.** Every rule that fires produces an `AgentFactor` with a human sentence and evidence links.
4. **Versioned.** `heuristic-v1.2.0` is stored on every decision. Changing a rule bumps the version; old decisions keep their old version and remain reproducible.
5. **Honest.** `ModelKind = "rules"`. No part of this system may describe itself as a learned model.

7.2 Rule interface

```

public interface IScoringRule
{
    string Key { get; }           // "completed_circles" - matches frontend AgentFactor.key
    string Label { get; }         // "Completed circles"
    string Trigger { get; }       // published on /agent as the rule's condition
    int Order { get; }
    AgentFactor? Evaluate(ScoringContext ctx); // null = rule did not fire
}

public sealed record ScoringContext(
    string Subject,
    Circle Circle,
    ReputationProfile Profile,
    int ActiveCircleCount,
    int WalletAgeDays,
    BigInteger LargestPotCompleted,
    DateTimeOffset? LastLatePaymentAt,
    IReadOnlyList<CompletedCircleRef> CompletedCircles,
    DateTimeOffset Now);

```

Rules are registered in DI and enumerated in `Order`. Adding a rule is one class and one registration line — no changes to the engine, the API, or the UI.

7.3 The published rule set (v1.2.0)

These must match frontend spec §9.5 exactly — that table is rendered from `/v1/agent/identity`.

Order	Key	Effect (bps)	Trigger
10	completed_circles	−1500 each, floor −3000	Circle finished with zero missed rounds
20	wallet_age	−1000	First on-chain activity older than 12 months
30	clean_streak	−500	6+ consecutive on-time rounds
40	circle_size_exposure	+2500	Pot per round more than 2× the largest circle completed
50	concurrent_obligations	+1500 per active circle beyond the first	
60	late_payments	+2000	Any late payment in the last 90 days
70	prior_default	+5000, and floors the final multiplier at 20000	Any default on record
80	new_wallet	+2500	First on-chain activity under 3 months
90	no_defaults	0 (neutral, informational)	No defaults on record

Sample implementation:

```

public sealed class CompletedCirclesRule : IScoringRule
{
    public string Key => "completed_circles";
    public string Label => "Completed circles";
    public string Trigger => "Circle finished with zero missed rounds";
    public int Order => 10;

    public AgentFactor? Evaluate(ScoringContext ctx)
    {
        var clean = ctx.CompletedCircles.Where(c => c.RoundsMissed == 0).ToList();
        if (clean.Count == 0) return null;

        var delta = Math.Max(-3000, clean.Count * -1500);

        return new AgentFactor(
            Key: Key,
            Label: Label,
            Detail: clean.Count == 1
                ? "You've finished 1 circle without missing a payment. That reduces your deposit."
                : $"You've finished {clean.Count} circles without missing a payment. That reduces your deposit.",
            Direction: FactorDirection.Decrease,
            DeltaBps: delta,
            Evidence: clean.Take(3).Select(c => new FactorEvidence(
                Label: c.Name,
                Value: $"{c.RoundsTotal}/{c.RoundsTotal} rounds on time",
                Href: $"/circles/{c.PublicId}")).ToList());
    }
}

```

Detail is user-facing copy in the join flow (frontend spec §9.9 step 2). Write it in second person, keep it to one sentence, and never use jargon. This is the sentence that makes a nervous saver understand why they're being asked for 300 USDC instead of 200.

7.4 The engine

```

public sealed class ScoringEngine(IEnumerable<IScoringRule> rules, IOptions<WhamOptions> opts)
{
    public ScoringResult Score(ScoringContext ctx)
    {
        var factors = rules.OrderBy(r => r.Order)
            .Select(r => r.Evaluate(ctx))
            .Where(f => f is not null)
            .Cast<AgentFactor>()
            .ToList();

        var baseline = ctx.Circle.BaseCollateralBps;
        var raw = baseline + factors.Sum(f => f.DeltaBps);

        // Floors and ceilings, applied in order
        var floor = factors.Any(f => f.Key == "prior_default") ? 20000 : 7500;
        var final = Math.Clamp(raw, floor, 30000);

        var collateral = ctx.Circle.ContributionPerRound * final / 10000;

        return new ScoringResult(
            Factors: factors,
            BaselineMultiplierBps: baseline,
            FinalMultiplierBps: (ushort)final,
            CollateralRequired: collateral,
            RiskBand: BandFor(final),
            ModelVersion: opts.Value.Agent.ModelVersion);
    }

    private static byte BandFor(int bps) => bps switch
    {
        <= 10000 => 0, // A
        <= 16000 => 1, // B
        <= 22000 => 2, // C
        _ => 3 // D
    };
}

```

The collateral formula, stated once (matches frontend spec §7.4):

```

finalMultiplierBps = clamp(baseCollateralBps + Σ factor.deltaBps, floor, 30000)
collateralRequired = contributionPerRound × finalMultiplierBps / 10000

```

Integer division truncates, which rounds the deposit *down*. That favours the member by at most one base unit and is the correct direction to be wrong in.

7.5 EIP-712 quote signing

The contract's `join()` verifies this signature against the circle's configured agent address (contract spec §8.3). The typed data must match the Solidity struct byte for byte.

```

[Struct("AgentQuote")]
public sealed class AgentQuoteMessage
{
    [Parameter("address", "subject", 1)] public string Subject { get; set; } = default!;
    [Parameter("address", "circle", 2)] public string Circle { get; set; } = default!;
    [Parameter("uint256", "collateralRequired", 3)] public BigInteger CollateralRequired { get; set; }
    [Parameter("uint8", "riskBand", 4)] public byte RiskBand { get; set; }
    [Parameter("uint256", "nonce", 5)] public BigInteger Nonce { get; set; }
    [Parameter("uint64", "expiresAt", 6)] public ulong ExpiresAt { get; set; }
}

public sealed class QuoteSigner(IOptions<WhamOptions> opts, IChainReader chain)
{
    private readonly EthEckey _key = new(opts.Value.Agent.PrivateKey);

    public async Task<SignedQuote> SignAsync(AgentDecision d, string circleAddress, CancellationToken ct)
    {
        // Nonce comes from the contract, not from us – it is the replay guard
        var nonce = await chain.GetAgentNonceAsync(circleAddress, d.Subject, ct);

        var typedData = new TypedData<Domain>
        {
            Domain = new Domain
            {
                Name = "WhamAgent",
                Version = "1",
                ChainId = opts.Value.ChainId,
                VerifyingContract = circleAddress
            },
            Types = MemberDescriptionFactory.GetTypesMemberDescription(
                typeof(Domain), typeof(AgentQuoteMessage)),
            PrimaryType = "AgentQuote"
        };

        var message = new AgentQuoteMessage
        {
            Subject = d.Subject,
            Circle = circleAddress,
            CollateralRequired = d.CollateralRequired,
            RiskBand = d.RiskBand,
            Nonce = nonce,
            ExpiresAt = (ulong)d.ExpiresAt.ToUnixTimeSeconds()
        };

        var signature = new Eip712TypedDataSigner()
            .SignTypedDataV4(message, typedData, _key);

        return new SignedQuote(signature, nonce, message);
    }
}

```

Verify the round-trip in a test against a locally deployed contract before you trust it. EIP-712 mismatches — a `uint64` where the contract has `uint256`, a field out of order, a domain name typo — produce a valid-looking signature that reverts with `BadAgentSignature()`. This is the single most likely thing to break on demo day, and it breaks silently until the user pays gas.

7.6 ERC-8004 attestation

Decisions are attested on-chain **asynchronously**, batched, after the quote is returned. The join flow must never block on a transaction the user isn't paying for.

```

public sealed class AttestationPublisher(IWeb3Provider web3, WhamDbContext db, IOptions<WhamOptions> opts)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using var timer = new PeriodicTimer(
            TimeSpan.FromMinutes(opts.Value.Agent.AttestationIntervalMinutes));

        while (await timer.WaitForNextTickAsync(ct))
        {
            var pending = await db.AgentDecisions
                .Where(d => d.AttestationTxHash == null)
                .OrderBy(d => d.CreatedAt)
                .Take(opts.Value.Agent.AttestationBatchSize)
                .ToListAsync(ct);

            if (pending.Count == 0) continue;

            // ref = keccak256(abi.encode(decisionId, subject, circle, collateral, finalBps, modelVersion))
            var refs = pending.Select(BuildAttestationRef).ToArray();
            var receipt = await reputationRegistry.SubmitBatchAsync(agentId, refs, ct);

            foreach (var d in pending)
            {
                d.AttestationTxHash = receipt.TransactionHash;
                d.AttestedAt = DateTimeOffset.UtcNow;
            }
            await db.SaveChangesAsync(ct);
        }
    }
}

```

The attestation ref is a hash, not the decision content — the full record lives at `attestationUri (/v1/agent/decisions/{id})`, public per frontend spec §9.6) and the hash proves it wasn't edited afterwards. Anyone can recompute the hash from the published JSON and check it against the chain. Document this on the `/agent` page; it's the whole auditability claim in one sentence.

7.7 Outcome resolution

An attestation with no outcome is a prediction nobody scored. The `OutcomeResolver` background job closes the loop:

```
// Runs hourly
foreach (var decision in db.AgentDecisions.Where(d => d.Outcome == DecisionOutcome.Pending))
{
    var member = await FindMemberAsync(decision.CircleId, decision.Subject);

    if (member is null && decision.ExpiresAt < now)
        decision.Outcome = DecisionOutcome.Expired; // quoted, never joined
    else if (member?.Status == MemberStatus.Defaulted)
        decision.Outcome = DecisionOutcome.Defaulted;
    else if (member?.Status == MemberStatus.Completed)
        decision.Outcome = member.RoundsMissed == 0
            ? DecisionOutcome.CompletedClean
            : DecisionOutcome.MissedPayments;

    if (decision.Outcome != DecisionOutcome.Pending)
    {
        decision.OutcomeResolvedAt = now;
        await attestations.PublishOutcomeAsync(decision, ct); // second ERC-8004 write
    }
}
```

Expired decisions are excluded from the accuracy stats — a quote nobody acted on tells you nothing about the agent’s judgement.

7.8 GET /v1/agent/stats

```
var resolved = await db.AgentDecisions
    .Where(d => d.Outcome != DecisionOutcome.Pending && d.Outcome != DecisionOutcome.Expired)
    .GroupBy(d => d.RiskBand)
    .Select(g => new
    {
        Band = g.Key,
        Decisions = g.Count(),
        CleanCompletions = g.Count(d => d.Outcome == DecisionOutcome.CompletedClean),
        MissedPayments = g.Count(d => d.Outcome == DecisionOutcome.MissedPayments),
        Defaults = g.Count(d => d.Outcome == DecisionOutcome.Defaulted)
    })
    .ToListAsync(ct);
```

Predicted default rates are a published constant per band, declared before outcomes exist. That’s what makes the comparison meaningful — a “prediction” fitted after the fact is not a prediction.

```
public static readonly IReadOnlyDictionary<byte, double> PredictedDefaultRate = new Dictionary<byte, double>
{
    [0] = 0.01, // A
    [1] = 0.03, // B
    [2] = 0.07, // C
    [3] = 0.14 // D
};
```

calibrationError = mean absolute difference between predicted and realised, weighted by decision count.

collateralCoverageRate = of all realised shortfalls, the share covered by the defaulter’s own posted collateral without touching the insurance pool. This is the number that says whether the underwriting actually worked.

7.9 Testing the agent

Table-driven, and the highest-value tests in the repo:

```
[Theory]
[InlineData(/* new wallet, starter circle */ 0, 0, 30, 20000, 22500, 3)] // → band D
[InlineData(/* 2 clean circles, 14mo wallet */ 2, 0, 420, 15000, 15000, 1)] // → band B
[InlineData(/* prior default */ 1, 1, 500, 15000, 20000, 2)] // floor applies
public void Scores_deterministically(
    int completed, int defaults, int walletAgeDays,
    ushort baseBps, ushort expectedFinalBps, byte expectedBand) { ... }
```

Plus: - **Determinism**: same context scored 100 times → identical output including factor order. - **Bounds**: no input combination produces a multiplier outside `[7500, 30000]`. - **Copy coverage**: every rule that can fire produces a non-empty `Detail` — a factor with a blank sentence renders as an empty card in the join flow. - **Signature round-trip**: sign a quote, recover the signer with `Eip712TypedDataSigner.RecoverFromSignatureV4`, assert it equals the configured agent address.

8. Chain integration

8.1 Web3 client

```
services.AddSingleton<IWeb3Provider>(sp =>
{
    var o = sp.GetRequiredService<IOptions<WhamOptions>>().Value;
    var web3 = new Web3(o.RpcUrl);
    web3.TransactionManager.UseLegacyAsDefault = false; // EIP-1559
    return new Web3Provider(web3, o.RpcUrlFallback);
});
```

Wrap every RPC call in a Polly pipeline: 3 retries with exponential backoff and jitter, then failover to `RpcUrlFallback`. Public Arbitrum Sepolia endpoints rate-limit, and they will do it during the demo. Configure a paid endpoint (Alchemy/Infura) as primary and the public one as fallback.

Read caching — the API reads chain state on hot paths (`balanceOf` on every authenticated circle list). Cache in `IMemoryCache`:

Call	TTL
<code>balanceOf(user)</code>	30s
<code>allowance(user, circle)</code>	15s
<code>config()</code>	permanent (immutable after deploy)
<code>totalReserves()</code>	60s
<code>agentNonce(circle, user)</code>	never cache — it's a replay guard

8.2 Event DTOs

```

[Event("MemberJoined")]
public sealed class MemberJoinedEventDto : IEventDTO
{
    [Parameter("address", "member", 1, true)] public string Member { get; set; } = default!;
    [Parameter("uint256", "collateral", 2, false)] public BigInteger Collateral { get; set; }
    [Parameter("uint8", "riskBand", 3, false)] public byte RiskBand { get; set; }
    [Parameter("bytes32", "decisionRef", 4, false)] public byte[] DecisionRef { get; set; } = default!;
}

[Event("RoundSettled")]
public sealed class RoundSettledEventDto : IEventDTO
{
    [Parameter("uint8", "round", 1, true)] public byte Round { get; set; }
    [Parameter("address", "winner", 2, true)] public string Winner { get; set; } = default!;
    [Parameter("uint16", "discountBps", 3, false)] public ushort DiscountBps { get; set; }
    [Parameter("uint256", "netPayout", 4, false)] public BigInteger NetPayout { get; set; }
    [Parameter("uint256", "redistributedPerMember", 5, false)] public BigInteger Redistributed { get; set; }
    [Parameter("uint256", "insuranceDrawn", 6, false)] public BigInteger InsuranceDrawn { get; set; }
}

```

One DTO per event in contract spec §8.3, plus `CircleCreated`, the reputation events, and the insurance pool events.

8.3 The indexer

Poll by **topic across all addresses**, then filter to known circles. Filtering by address in the RPC call means rebuilding the filter every time a circle deploys, and hitting per-request address-list limits once you have a few dozen circles.

```

public sealed class ChainIndexer(IWeb3Provider web3, IServiceScopeFactory scopes,
                                IOption<WhamOptions> opts, ILogger<ChainIndexer> log)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromSeconds(opts.Value.Indexer.PollIntervalSeconds));

        while (await timer.WaitForNextTickAsync(ct))
        {
            try { await TickAsync(ct); }
            catch (Exception ex) { log.LogError(ex, "Indexer tick failed"); }
        }
    }

    private async Task TickAsync(CancellationToken ct)
    {
        await using var scope = scopes.CreateAsyncScope();
        var db = scope.ServiceProvider.GetRequiredService<WhamDbContext>();

        var checkpoint = await db.IndexerCheckpoints.SingleAsync(c => c.Name == "main", ct);
        var head = (long)(await web3.Eth.Blocks.GetBlockNumber.SendRequestAsync()).Value;
        var safeHead = head - opts.Value.Indexer.ConfirmationDepth;

        var from = checkpoint.LastProcessedBlock + 1;
        if (from > safeHead) return;
        var to = Math.Min(safeHead, from + opts.Value.Indexer.MaxBlockRange - 1);

        var logs = await web3.Eth.Filters.GetLogs.SendRequestAsync(new NewFilterInput
        {
            FromBlock = new BlockParameter((ulong)from),
            ToBlock    = new BlockParameter((ulong)to),
            Topics     = [KnownTopics.All]    // topic0 OR-set across every Wham event
        });

        var watched = await db.Circles.Where(c => c.OnchainAddress != null)
            .Select(c => c.OnchainAddress!).ToHashSetAsync(ct);
        watched.Add(opts.Value.Contracts.CircleFactory.ToLowerInvariant());
        watched.Add(opts.Value.Contracts.Reputation.ToLowerInvariant());
        watched.Add(opts.Value.Contracts.InsurancePool.ToLowerInvariant());

        foreach (var l in logs.Where(l => watched.Contains(l.Address.ToLowerInvariant()))
            .OrderBy(l => l.BlockNumber.Value)
            .ThenBy(l => l.LogIndex.Value))
        {
            await ProcessAsync(db, l, ct);    // idempotent; see §8.4
        }

        checkpoint.LastProcessedBlock = to;
        checkpoint.UpdatedAt = DateTimeOffset.UtcNow;
        await db.SaveChangesAsync(ct);
    }
}

```

8.4 Idempotency and ordering

Every log is processed inside one transaction that also inserts the `chain_events` row. The unique index on `(chain_id, tx_hash, log_index)` makes reprocessing free:

```

private async Task ProcessAsync(WhamDbContext db, FilterLog log, CancellationToken ct)
{
    await using var tx = await db.Database.BeginTransactionAsync(ct);
    try
    {
        db.ChainEvents.Add(ChainEvent.From(log));
        await db.SaveChangesAsync(ct); // throws 23505 if already processed
    }
    catch (DbUpdateException e) when (e.IsUniqueViolation())
    {
        await tx.RollbackAsync(ct);
        return; // already handled – nothing to do
    }

    await handlers.DispatchAsync(db, log, ct); // projection writes
    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
}

```

Logs are processed in `(blockNumber, logIndex)` order, which is the chain’s own ordering. Never parallelise this loop — `MemberJoined` before `ContributionPaid` matters.

8.5 Reorg handling

Arbitrum reorgs are rare but not impossible, and a sequencer reorg during judging would be memorable for the wrong reasons.

On each tick, verify the block hash of the last processed block still matches what the node reports:

```

var stored = checkpoint.LastProcessedBlockHash;
var actual = (await web3.Eth.Blocks.GetBlockWithTransactionsHashesByNumber
    .SendRequestAsync(new BlockParameter((ulong)checkpoint.LastProcessedBlock)))
    .BlockHash;

if (stored is not null && !string.Equals(stored, actual, StringComparison.OrdinalIgnoreCase))
{
    log.LogWarning("Reorg detected at block {Block}. Rewinding {N} blocks.",
        checkpoint.LastProcessedBlock, opts.Value.Indexer.ReorgRewindBlocks);

    var rewindTo = checkpoint.LastProcessedBlock - opts.Value.Indexer.ReorgRewindBlocks;
    await db.ChainEvents.Where(e => e.BlockNumber > rewindTo).ExecuteDeleteAsync(ct);
    await projections.RebuildFromAsync(rewindTo, ct);
    checkpoint.LastProcessedBlock = rewindTo;
    return;
}

```

`RebuildFromAsync` reprocesses retained `chain_events` rows. Because every projection write is derived purely from events, a rewind-and-replay is always correct. This is the payoff for the “the indexer is the only writer” rule in §6.6.

8.6 Full rebuild command

```
dotnet run --project src/Wham.Api -- reindex --from 91450000 --truncate-projections
```

Truncates `circles` projected fields, `circle_members`, `rounds`, `contributions`, `bids`, `insurance_claims`, and `chain_events`, then replays from the factory deployment block. Preserves `users`, `agent_decisions`, `notifications`.

Run this once during week 2 to prove it works. If it doesn’t, you have a hidden write path outside the indexer — find it before demo week, not during.

8.7 Handler → projection map

Event

Projection effect

CircleCreated	Link <code>OnchainAddress</code> , verify <code>config()</code> against draft, add to watch set
MemberJoined	Upsert <code>circle_members</code> , <code>MemberCount++</code> , mark reservation consumed, link decision
MemberLeft	Status → <code>Exited</code> , <code>MemberCount--</code> , release seat
ContributionPaid	Insert <code>contributions</code> , <code>RoundsPaid++</code> , <code>ContributionsReceived++</code> , cancel that reminder
BidPlaced	Insert <code>bids</code> , demote prior leader to <code>outbid</code> , notify the outbid member
BidWithdrawn	Mark bid <code>withdrawn</code> , recompute leader
RoundSettled	Round → <code>Settled</code> , winner fields, member → <code>PaidOut</code> , open next round, notify all
CollateralDrawn	<code>CollateralSlashed +=</code> , member → <code>Delinquent</code> , <code>RoundsMissed++</code> , notify urgently
PayoutClaimed	<code>PayoutAmountNet</code> , notify
CollateralReleased	Member → <code>Completed</code> , <code>CollateralReleasedAt</code>
CircleCompleted	Circle → <code>Completed</code> , trigger reputation recompute for all members
ClaimPaid (pool)	Insert <code>insurance_claims</code> , <code>InsuranceDrawn +=</code>
ScoreUpdated (rep)	Update <code>reputation_profiles</code> , insert <code>reputation_events</code>

9. Reputation engine

Score is computed by the backend and written on-chain by the contract on circle completion. The backend's copy is a projection; the chain's is authoritative. When they diverge, trust the chain and log loudly.

9.1 Score movements

Published on `/app/reputation` (frontend spec §9.16) — so these constants are product copy:

Event	Delta
Round paid on time	+8
Round paid late	−15
Round missed	−60
Circle completed clean	+120
Circle completed with misses	+40
Default	−400

Score clamps to `[0, 1000]`. Starting score for a new wallet is 0.

9.2 Tier thresholds

Tier	Requires
Starter	Default
Standard	1 completed circle and score ≥ 250
Trusted	3 completed circles and score ≥ 600 and zero defaults

`nextTier` on the profile computes the gap in both dimensions and returns human strings for the unlock list.

9.3 Recompute

Triggered by the indexer on any event that moves a member's record. Full recompute from `reputation_events` rather than incremental mutation — the event log is small, and a recomputable score is a debuggable score.

10. Notifications

10.1 Channels

Channel	Transport	Always on
In-app	<code>notifications</code> table, polled by the client	Yes
Email	Postmark/Resend, templated	Opt-in
Telegram	Bot API <code>sendMessage</code> to stored <code>chat_id</code>	Opt-in

Telegram matters disproportionately for the target user — it’s the default messaging app across much of the diaspora audience, and email deliverability to those regions is unreliable. Build the bot connection flow (one-time code → `/start` → store `chat_id`) in week 3, not as a stretch goal.

10.2 Scheduling

When the indexer opens a round, it schedules reminders for every member from their `NotifyPaymentDueHours`:

```
foreach (var member in activeMembers)
foreach (var leadHours in member.User.Preferences.NotifyPaymentDueHours)
{
    var sendAt = round.ContributionDueAt.AddHours(-leadHours);
    if (sendAt <= now) continue;

    db.ScheduledReminders.Add(new ScheduledReminder
    {
        Address = member.Address,
        Kind = NotificationKind.PaymentDue,
        CircleId = circle.Id,
        RoundIndex = round.Index,
        DedupeKey = $"payment_due:{circle.Id}:{round.Index}:{member.Address}:{leadHours}",
        SendAt = sendAt
    });
}
```

`DedupeKey` is uniquely indexed. Re-running the scheduler after a reorg replay inserts nothing new.

Cancel on payment. The `ContributionPaid` handler must cancel that member’s outstanding reminders for that round. Nothing erodes trust faster than “your payment is due” arriving three hours after they paid.

```
await db.ScheduledReminders
    .Where(r => r.DedupeKey.StartsWith($"payment_due:{circleId}:{roundIndex}:{address}"))
    && r.SentAt == null)
    .ExecuteUpdateAsync(s => s.SetProperty(r => r.CancelledAt, now), ct);
```

10.3 Dispatch

`ReminderDispatcher` runs every 5 minutes: claim due reminders with `FOR UPDATE SKIP LOCKED`, re-validate the obligation still stands, render per locale and timezone, send, record delivery. Failures retry 3× with backoff, then mark `failed` — never block the queue on one bad address.

10.4 Copy

Templates live in `Wham.Infrastructure/Notifications/Templates/` as `{kind}.{locale}.liquid`. Rules:

- **State the amount and the deadline in the first line.** “50.00 USDC due to Diaspora Circle #3 in 24 hours.”
- **State the consequence when there is one.** “If it isn’t paid by 9 Oct, 50.00 USDC will be taken from your deposit.”
- Deep-link straight to the action, not the homepage.
- Render times in the user’s stored timezone with the UTC deadline noted.
- Persian templates get RTL-safe formatting and Latin numerals for money.

11. Background services

Service	Interval	Role	Job
ChainIndexer	4s	indexer	Poll logs, project state
RoundLifecycleService	1 min	workers	Open bidding, close bidding, advance round status
SettlementKeeper	1 min	workers	Call <code>settleRound</code> when due
ReminderDispatcher	5 min	workers	Send scheduled notifications
SeatReservationSweeper	1 min	workers	Release expired reservations
AttestationPublisher	10 min	workers	Batch ERC-8004 decision attestations
OutcomeResolver	1 hour	workers	Resolve decision outcomes, publish outcome attestations
StatsAggregator	5 min	workers	Materialise <code>/v1/stats</code>
CircleStarter	1 min	workers	Start circles that reached quorum

11.1 The settlement keeper

Frontend spec §11.2 flags this as an open decision. **Resolve it this way:** the backend keeper is primary, with a member-facing “Settle round” button as fallback.

```
var due = await db.Rounds
    .Where(r => r.Status == RoundStatus.Funding
        && r.ContributionDueAt.AddMinutes(opts.SettleGraceMinutes) < now
        && r.SettledAt == null
        && r.SettleAttempts < 5)
    .Include(r => r.Circle)
    .ToListAsync(ct);

foreach (var round in due)
{
    var gasPrice = await web3.Eth.GasPrice.SendRequestAsync();
    if (Web3.Convert.FromWei(gasPrice, UnitConversion.EthUnit.Gwei) > opts.MaxGasPriceGwei)
    {
        log.LogWarning("Skipping settle for {Circle} round {Round}: gas {Gwei} gwei", ...);
        continue;
    }

    try
    {
        var receipt = await circleService.SettleRoundAsync(round.Circle.OnchainAddress!, round.Index, ct);
        round.SettleAttemptedAt = now;
        round.SettleAttempts++;
        // Do NOT write settled state here – the RoundSettled event will do it (§6.6)
    }
    catch (Exception ex)
    {
        round.SettleAttempts++;
        round.SettleFailureReason = ex.Message;
        if (round.SettleAttempts >= 5) await alerts.RaiseAsync($"Settlement stuck: {round.Id}", ct);
    }
}
```

The keeper uses its **own** key with its own funded balance, separate from the agent key (§2.4). Monitor that balance — a keeper that runs out of gas stalls every circle simultaneously, and it will happen at 2am on demo eve. Add a startup check that logs the keeper balance and an hourly warning below 0.01 ETH.

11.2 Circle starter

A `Forming` circle with `MemberCount == MemberTarget` and `StartsAt <= now` transitions to `Active`. If `StartsAt` passes without quorum, the policy is: extend once by the round duration, then cancel and let members withdraw. Notify on both.

12. Testing

Layer	Tool	What
Scoring engine	xUnit, table-driven	Every rule, every boundary, determinism, bounds (\$7.9)
EIP-712	xUnit + Anvil	Sign → deploy → <code>join()</code> succeeds; tamper a field → reverts
Indexer	xUnit + Anvil	Replay fixtures, idempotency (process twice, assert identical state), reorg rewind
API	<code>WebApplicationFactory</code> + <code>Testcontainers</code> <code>Postgres</code>	Auth guards, error envelopes, the anonymous/authenticated cache split (\$3.7)
Concurrency	Integration	20 parallel <code>join-intent</code> calls on a 1-seat circle → exactly 1 reservation

The concurrency test and the idempotency test are the two that will actually catch bugs. Write them first.

13. Deployment

`docker-compose.yml`: `postgres`, `api` (×1), `indexer` (×1, **never scale beyond one**), `workers` (×1), `caddy` for TLS.

```
services:
  api:      { environment: [WHAM_ROLE=api],      deploy: { replicas: 2 } }
  indexer:  { environment: [WHAM_ROLE=indexer],  deploy: { replicas: 1 } }
  workers:  { environment: [WHAM_ROLE=workers],  deploy: { replicas: 1 } }
```

Only the `workers` container gets `Wham:Agent:PrivateKey` and `Wham:Keeper:PrivateKey`. The `api` container has no signing keys at all — which means a compromised API cannot underwrite or settle anything, and you can say so on the agent page.

Health checks: `/health/live` (process up), `/health/ready` (DB reachable, RPC reachable, indexer lag under 60 blocks). Expose indexer lag as a metric; it's the one number that tells you the whole system is healthy.

14. Build order

Week 1 — Foundations

#	Task
1	Solution scaffold, EF Core, all entities and migrations (\$4)
2	Error envelope, serialisation conventions, <code>EvmAddress</code> , <code>BigInteger</code> converters (\$3)
3	SIWE nonce/verify/JWT, all five checks (\$5)
4	Seed data command: 6 circles across statuses matching the frontend mocks exactly
5	<code>GET /v1/circles</code> , <code>GET /v1/circles/{id}</code> , viewer context (\$6.1–6.2)
6	Nethereum wiring, event DTOs, read caching (\$8.1–8.2)

Week 2 — The agent and the indexer

#	Task
7	Scoring engine + all nine rules + tests (\$7.2–7.4) — before anything

that consumes it

8	EIP-712 signer + round-trip test against Anvil (\$7.5)
9	Indexer: polling, idempotency, handler map (\$8.3-8.4, \$8.7)
10	Join flow: <code>join-intent</code> , reservations, capacity locking, <code>confirm-join</code> (\$6.5-6.6)
11	Circle create + <code>link-deployment</code> with config verification (\$6.3-6.4)
12	Reputation engine + <code>/me/reputation</code> (\$9)
13	Reorg handling + full reindex command, proven working (\$8.5-8.6)

Week 3 — Closing the loop

#	Task
14	Attestation publisher + outcome resolver + <code>/v1/agent/*</code> (\$7.6-7.8)
15	Settlement keeper + round lifecycle (\$11)
16	Notifications: in-app, email, Telegram, scheduling, cancellation (\$10)
17	Obligations, <code>/me/summary</code> , <code>/stats</code> aggregator (\$6.7-6.9)
18	Insurance pool endpoints
19	Rate limits, caching policy, health checks
20	Seed the demo chain state (see below)

14.1 Demo data seeding

Write this as a command, not a manual process, because you will run it more than once:

```
dotnet run --project src/Wham.Api -- seed-demo --wallets 10 --fund
```

It should produce, on Arbitrum Sepolia:

- 8 funded wallets, all with SIWE-able keys you control
- One circle **mid-rotation**: 3 rounds settled, round 4 in bidding with two competing bids
- One member **delinquent** with a visible collateral draw
- One settled round showing real discount redistribution
- Two agent decisions with **visibly different multipliers** — a fresh wallet at 2.25× beside a two-circle veteran at 1.5×. That side-by-side is the pitch in one screenshot.
- At least one **resolved** decision with an outcome, so `/agent` has real calibration data
- One insurance claim paid

14.2 Open decisions carried from the frontend spec

#	Question	Recommendation
1	Collateral top-up for delinquent members	Add <code>topUpCollateral()</code> to the contract — it's ~15 lines and turns a dead-end into a recovery path. If it doesn't land, cut the frontend CTA.
2	Who calls <code>settleRound</code>	Backend keeper primary, member button fallback (\$11.1). Decided.
3	Bid ties	Earliest bid wins. Contract must enforce strict improvement (<code>></code> , not <code>>=</code>), which makes ties impossible on-chain — confirm this and simplify the backend.
4	Payout-then-default	The contract must lock the paid-out member's full remaining obligation against their deposit at payout time. Confirm; if it doesn't, the discount bidding mechanic has an exploit and the fix is a contract change, not a backend one.
5	Invite-code join	Add <code>inviteCode</code> to the <code>join-intent</code> request

body; validate against `circles.invite_code`
before the capacity check.

Items 1 and 4 are contract changes. Raise them with your Solidity teammate in week 1, not week 3.